

Situation engineering for evidence-applicable artificial intelligence

Geunsik Lim ^{1*}

¹ Data Service Lab., Samsung Electronics, Seoul, South Korea .

Corresponding author(s). E-mail(s): geunsik.lim@samsung.com;

Abstract

Artificial-intelligence systems increasingly combine prompts, retrieved context, memory, tools and agent harnesses, yet these components do not by themselves determine whether an available fact applies to the question being answered. A statement may be well supported and still be wrong because it is stale, merely proposed, outside the relevant scope, unknown to the queried observer or true only in another possible world. We call this failure *situation blindness* and define *situation engineering* as the explicit construction, updating, validation and exposure of the query-relative world state that governs evidence applicability. We introduce Situation-Catching Question Answering (SCQA), a reference control architecture that separates situation sensing, state update and response policy, and SituationCatch-Bench, a 4,200-instance diagnostic suite covering temporal change, hidden premises, source conflict, modality, scope, observer knowledge and counterfactual worlds. Under oracle structured states, SCQA satisfies all generated invariants, whereas non-situational rules fail in predictable categories. We interpret this result as a software-level constructive upper bound, not as evidence of end-to-end language-model performance. The main contribution is therefore a falsifiable problem definition, a modular evaluation decomposition and a reproducible research scaffold for measuring where situation sensing, state maintenance, policy selection and generation fail. By locating situation engineering alongside prompt, context, retrieval, memory, tool, harness, evaluation and serving engineering, the framework provides researchers with interfaces, reporting standards and open experiments for developing systems that distinguish relevant evidence from evidence that is actually applicable.

Keywords: large language models, situation engineering, evidence applicability, situation awareness, hallucination, trustworthy artificial intelligence, temporal reasoning, question answering, agent systems

Large language models (LLMs) increasingly answer questions by combining parametric knowledge with retrieved documents, tools, memory and long conversational histories. These mechanisms improve evidence coverage, but they do not guarantee that a system identifies the *situation* in which a fact is valid. A model may read that a person was appointed and later resigned, yet still name that person as the current office holder. It may turn a proposal into a decision, extend a local policy to an entire organization, answer from information unavailable to the queried observer, or collapse a hypothetical scenario into the real world. In each case the decisive evidence can already be present. The failure is not simply missing knowledge; it is incorrect evidence applicability.

We define *situation blindness* as producing an answer or action that is supported by at least one available claim but inconsistent with the time, scope, modality, observer knowledge or possible world required by the query. This category matters because conventional factuality checks can mark the supporting claim as true while missing that it is true in the wrong situation. Natural question-answering resources already show that a material subset of ordinary questions depends on extra-linguistic time or place, and that updated evidence does not eliminate the resulting performance loss [6, 21–24].

Retrieval-augmented generation supplies external evidence [1]; self-reflective and tool-using systems add critique and action [2–4]; context engineering organizes the inference-time information payload [16]; and agent harnesses mediate tools, memory, permissions, verification and feedback [17, 18]. These layers are necessary, but none requires an explicit answer to a separate question: *which interpretation of the available evidence governs this interaction now?*

We call the corresponding discipline *situation engineering*: the systematic design of representations, sensing interfaces, update rules, validation procedures and response policies that establish the active query-relative world state before an AI system commits to an answer or action. For a query q and evidence E , a minimal state is

$$S(q, E) = \{\mathcal{A}, \mathcal{V}, \mathcal{T}, \mathcal{P}, \mathcal{M}, \mathcal{K}, \mathcal{W}, \mathcal{U}\}, \quad (1)$$

where the variables denote actors, events, valid time, scope, modality and source status, observer-specific knowledge, possible world and unresolved answer-critical information. A policy then selects

$$\pi(S) \in \{\text{ANSWER, CLARIFY, RETRIEVE, ABSTAIN, ACT}\}. \quad (2)$$

This decomposition separates evidence availability from evidence applicability.

The present paper makes four bounded contributions. First, it defines situation blindness as a testable failure class and relates it to established work on situations, temporal reasoning, belief state tracking and knowledge revision. Second, it specifies situation engineering as a systems contract connecting prompt, context, retrieval, memory, tool, harness, evaluation and serving engineering without subsuming them. Third, it introduces SCQA and SituationCatch-Bench as a transparent oracle-state diagnostic for causal unit tests and component ablations. Fourth, it provides a staged evaluation and reporting protocol for future end-to-end studies on natural language, multiple models, human clarification and deployed agents. We deliberately do not

claim that the current rule-based implementation solves situation sensing in unrestricted text. Instead, the paper separates the solved control invariants from the open perception, learning and generalization problems so that future work can measure progress without conflating them.

Related conceptual foundations

Situation engineering draws on several mature traditions but is not identical to any one of them. Situation semantics treats meaning relative to partial situations [19]. Situation and event calculi formalize how actions change fluents over time [31–33]. Temporal reasoning provides interval relations and validity semantics [13]. Belief revision and truth-maintenance research asks how an agent should update inconsistent commitments [34, 35]. POMDPs and dialogue-state tracking maintain uncertainty over latent states for sequential decisions [36, 37]. Temporal knowledge graphs and bitemporal databases record changing facts and distinguish when a fact is valid from when it is stored [38].

The proposed contribution is a systems-level applicability contract for generative and agentic AI. It requires these representations to connect to natural-language evidence, provenance, unresolved alternatives and a response policy that can answer, retrieve, clarify, abstain or act. Thus the novelty is not a new universal logic. It is the explicit decomposition of modern AI reliability into (i) sensing a situation from heterogeneous evidence, (ii) maintaining a provenance-linked state, (iii) validating applicability and (iv) propagating unresolved state into behaviour. This framing allows existing logical, probabilistic and neural methods to be compared through a common set of interfaces and failure measurements.

From prompt engineering to situation engineering

The reliability stack surrounding foundation models has expanded in response to successive failure modes. Prompt engineering shapes the immediate instruction: what task to perform, which role to assume and how to express the output. Context engineering expands the unit of design to the information payload, including retrieval, memory, examples, tool observations and context-window management [16]. Harness engineering expands it again to the runtime substrate that mediates actions, permissions, verification, persistence and feedback [17, 18]. These layers are complementary, but none necessarily constructs an explicit answer to a different question: *which interpretation of the available evidence governs the present interaction?*

We use *situation engineering* for the systematic design of representations, update rules, sensing processes and decision policies that establish the active world state before an AI system commits to an answer or action. The term is not intended to replace context, memory, retrieval, tools, evaluation or harness engineering. It identifies the semantic control plane between information availability and justified behaviour. Context engineering can place an appointment notice and a later resignation in the same window. Retrieval engineering can find both. Memory engineering can retain both across sessions. Tool engineering can expose an authoritative registry. Harness engineering can guarantee that the records are fetched, logged and checked. Evaluation

Table 1 Relationship between neighbouring research areas and situation engineering. The entries describe typical emphasis rather than hard exclusions.

Area	Primary object	Typical strength	Common omission for generative AI	Situation-engineering interface
Situation/event calculus	Fluents and actions	Formal state transition	Natural-language sensing and provenance-aware response policy	Supplies transition semantics
Belief revision / truth maintenance	Consistent commitments	Conflict and retraction	Query-relative scope, observer and tool action	Supplies update and conflict rules
Dialogue-state tracking / POMDPs	Latent interaction state	Uncertainty and sequential policy	General factual validity and multiple possible worlds	Supplies probabilistic state tracking
Temporal KG / bitemporal data	Time-indexed facts	Validity and transaction history	Clarification, abstention and generated-claim audit	Supplies state storage and temporal retrieval
RAG / agent systems	Evidence and actions	Information access and task completion	Applicability of retrieved evidence	Supplies observations and capabilities
Situation engineering	Query-relative applicable state	Cross-layer applicability control	End-to-end sensing remains open	Integrates state, provenance and behaviour

engineering can detect that the final answer is wrong. Situation engineering is the layer that determines, before commitment, that the resignation supersedes the appointment for a question about the current office holder.

A layered taxonomy of AI systems engineering

The emerging vocabulary of “engineering” around generative AI mixes design objects that operate at different levels. Treating every term as a peer produces both overlap and conceptual inflation. We therefore organize the stack into three interacting planes (Fig. 1). The *semantic and cognitive plane* determines instructions, evidence, memory and the active situation. The *action and runtime plane* determines which capabilities can be invoked and how multi-step execution is governed. The *measurement and resource plane* determines how behaviour is evaluated and how inference is delivered under latency, throughput, cost and energy budgets. This grouping is functional rather than organizational: one production team or one software framework may implement several planes.

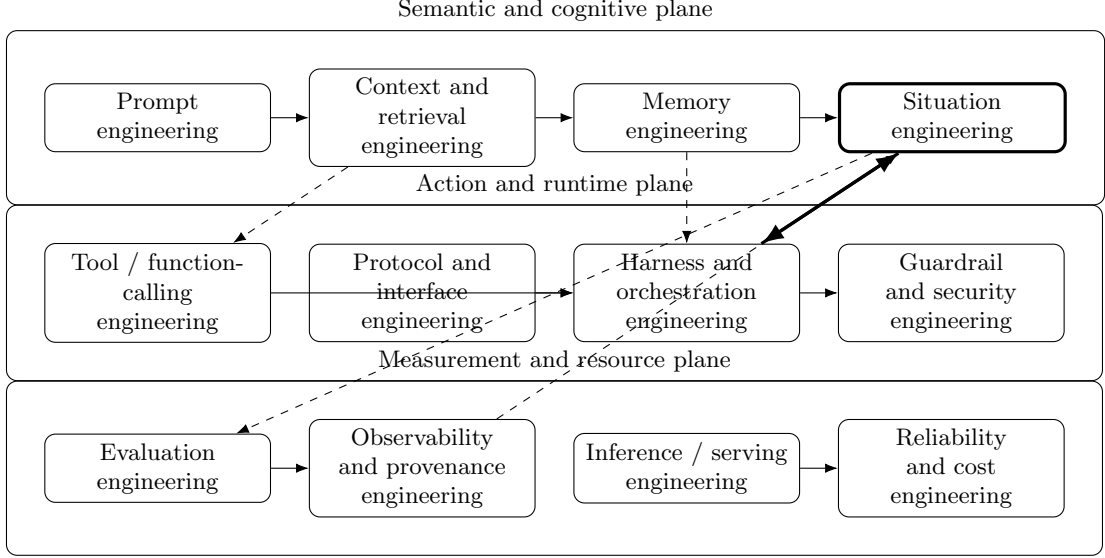


Fig. 1 A functional taxonomy of engineering disciplines surrounding modern AI systems. Solid horizontal arrows indicate common within-plane dependencies; dashed arrows indicate cross-plane feedback. Situation engineering is distinguished by its responsibility for evidence applicability and active-world-state control, not by ownership of a particular software component.

Several relationships are especially important. Retrieval engineering is a specialized part of context acquisition, but also has its own ranking, indexing, freshness and systems questions. Memory engineering stores and recovers information, whereas situation engineering decides whether a recovered memory is still valid, is superseded, belongs to the current user or world, or should remain only a competing hypothesis. Tool engineering specifies capabilities and invocation contracts; harness engineering composes those capabilities into guarded workflows. Protocols such as the Model Context Protocol standardize how prompts, resources and tools are exposed, but a protocol does not by itself determine the semantic applicability of the data it transports [26]. Evaluation engineering is not merely a final score: it is the sensing system for the engineering lifecycle, translating desired behaviour into tests, traces and regression signals [27, 28]. Inference and serving engineering are orthogonal but consequential because resource pressure can alter truncation, retrieval depth, model choice and verification budgets, thereby indirectly changing situation quality [29].

Situation engineering as an evidence-applicability discipline

A situation is not a longer prompt and not merely a collection of metadata. It is a constrained interpretation of evidence relative to a query. Let E be the available evidence and q a query. Context selection produces a subset or representation $C(q, E)$. Memory contributes retained observations M_t . Tools produce new observations O_t , and a harness supplies an execution trace H_t . Situation construction produces an active

state $S_t(q, C, M_t, O_t, H_t)$ containing the variables that decide whether each claim is applicable. Generation should therefore be conditioned not only on evidence relevance but on an applicability relation

$$\text{Apply}(e, q, S_t) \in \{\text{valid}, \text{invalid}, \text{unresolved}\}. \quad (3)$$

A reliable system must preserve the third value. Converting unresolved applicability into a plausible binary judgement is one mechanism by which fluent systems hallucinate.

We distinguish six operations. *Situation sensing* extracts events, actors, times, scopes, source status, perspectives and world identifiers from language, databases, tools or sensors. *Situation assembly* links those observations into candidate states. *Situation updating* applies event and belief transitions as new evidence arrives. *Situation validation* checks consistency, provenance, freshness and unresolved alternatives. *Situation policy* selects whether to answer, retrieve, clarify, act or abstain. Finally, *situation observability* records which state and evidence justified the decision. SCQA operationalizes a minimal version of these operations for question answering.

Why model scale, tools and longer context are insufficient

Increasing model capacity may improve event extraction and implicit reasoning, but it does not create a contract that the active state must be represented, updated and inspected. Longer windows can contain more mutually incompatible states: old and new policies, proposals and final decisions, public facts and private beliefs, or real and hypothetical scenarios. More tools can expand both competence and the consequences of applying an incorrect state. More memory can preserve both useful commitments and obsolete assumptions. The reliability objective is therefore not to maximize information volume or capability count, but to minimize answer-critical unresolved applicability subject to retaining the evidence and actions needed for the task.

A compact systems formulation is

$$\begin{aligned} \text{trustworthy outcome} = & \text{task alignment} + \text{evidence coverage} + \text{state continuity} \\ & + \text{capability correctness} + \text{runtime control} + \text{validated situation state} \\ & + \text{measurement and auditability} + \text{resource feasibility}. \end{aligned} \quad (4)$$

These terms map respectively to prompt, context/retrieval, memory, tool, harness/security, situation, evaluation/observability and serving engineering. The expression is a design decomposition rather than an additive statistical model: failure in any term can dominate the outcome.

Situation engineering as the coordinator of state-bearing components

Situation engineering has particularly close interfaces with four neighbouring disciplines. First, retrieval should be driven by missing situation variables, not only

semantic similarity. A system uncertain about whether a policy is active should retrieve enactment, effective-date and repeal evidence rather than more documents that merely discuss the policy. Second, memory should be event- and provenance-aware. It should store when a fact was learned, when it was valid, which agent observed it and what later event superseded it [30]. Third, tools should declare state effects and preconditions, analogous to typed transactions: a booking tool should report whether a reservation is held, confirmed, expired or cancelled, not merely return free text. Fourth, harnesses should enforce situation invariants before consequential action, for example blocking payment when identity, amount, authorization scope or transaction world remains unresolved.

This coordinator role does not make situation engineering a universal container for all other engineering. It owns the semantics of the active state and the policy consequences of uncertainty in that state. The retriever still owns ranking quality; the memory subsystem still owns retention and forgetting; the tool layer still owns safe invocation; the harness still owns workflow integrity; the evaluator still owns valid measurement; and the serving layer still owns resource efficiency. A useful implementation contract is therefore a typed interface rather than organizational consolidation:

$$(C_t, M_t, O_t, H_t) \rightarrow (\mathcal{S}_t, U_t, P_t), \quad (5)$$

where $\mathcal{S}_t$ is a set of candidate situations, U_t their uncertainty and unresolved variables, and P_t the permitted response or action policy.

A maturity model for situation-engineered AI

We propose five levels that turn the concept into a falsifiable systems agenda. At SE0, the system has no explicit situation representation and relies on language-model inference alone. At SE1, it tags isolated variables such as time or location. At SE2, it maintains an explicit state with provenance and update rules. At SE3, it detects missing variables, competing hypotheses and contradictions, and can clarify or abstain. At SE4, it learns and audits situation states across modalities, agents and long-running environments while quantifying uncertainty and distribution shift. SituationCatch-Bench primarily tests the transition from SE0/SE1 to SE2/SE3 under controlled conditions.

This maturity model prevents a common category error. A system that prints a JSON field called “situation” is not necessarily situation-aware. Evidence is required that the representation causally changes behaviour in the predicted cases, that ablations recover matching failure signatures, and that uncertainty in the state propagates to uncertainty in the answer. The experiments below provide this causal diagnostic for seven variables, while leaving natural-text sensing and SE4 operation as open problems.

Research agenda for epistemically regulated AI

A useful reliability target is not zero error but appropriate epistemic regulation. People ask for clarification, track who knows what, revise beliefs, recognize hypotheticals

and withhold judgement when the situation is underspecified. A useful computational target is therefore *human-like epistemic regulation*: the ability to identify which state is active, notice when it cannot be determined and choose a proportionate response. Progress requires connected research programmes across the engineering planes.

Neural situation sensing must extract implicit and distributed state variables from natural text, speech, images and operational traces. Dynamic state models must represent valid time, transaction time, event supersession, causal dependencies and observer-relative beliefs without erasing uncertainty. Active information acquisition must select the retrieval, memory recall, tool call or clarification that most reduces answer-critical uncertainty. Situation- conditioned calibration must estimate risk from state completeness, conflict and provenance in addition to model logits. Evaluation engineering must separately score sensing, updating, policy and final response under adversarially stale or manipulated evidence. Serving research must study how latency and cost budgets change the achievable depth of verification and state maintenance.

These programmes suggest that situation engineering can serve as a common semantic interface among otherwise separate communities studying RAG, temporal reasoning, knowledge graphs, agent memory, tools, protocols, agent harnesses, uncertainty, human–AI interaction, serving systems and AI safety. An event- memory system can expose a situation-state API; a retriever can optimize for missing variables; a tool can publish preconditions and state transitions; a harness can prevent action until state invariants pass; an evaluator can probe state-specific counterfactuals; and a generator can cite the state and evidence on which each claim depends. The practical goal is not a new label for existing components, but an auditable contract: no consequential answer or action should be produced without an explicit account of the situation in which it is valid.

Results

A controlled benchmark isolates seven applicability invariants

SituationCatch-Bench v0.1 contains 4,200 generated English instances, with 600 instances in each of seven diagnostic categories: temporal state change, omitted answer-critical premises, source conflict, proposed versus confirmed events, local versus universal scope, observer-dependent knowledge and real versus counterfactual worlds. Every item provides a question, shuffled claims, query time, required state variables, a gold action and, where answerable, a gold answer. The benchmark is intentionally transparent: each category is a software-level test of one applicability invariant rather than a claim to natural linguistic diversity.

Oracle-state control satisfies the generated invariants

SCQA was compared with lexical retrieval, recency-ranked retrieval and a latest-mention rule. All methods received the same generated claims; SCQA alone used the complete oracle situation state and transition semantics. SCQA obtained 100.0% action-and-answer accuracy. Latest mention obtained 85.7%, recency ranking 71.4% and lexical retrieval 70.0% (Fig. 3). These numbers are deterministic consequences

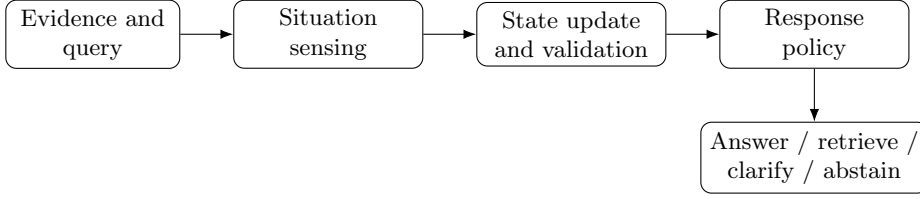


Fig. 2 Evaluation decomposition. The present experiment supplies oracle structured situation variables and evaluates state update and response policy. End-to-end research must additionally measure natural-language situation sensing.

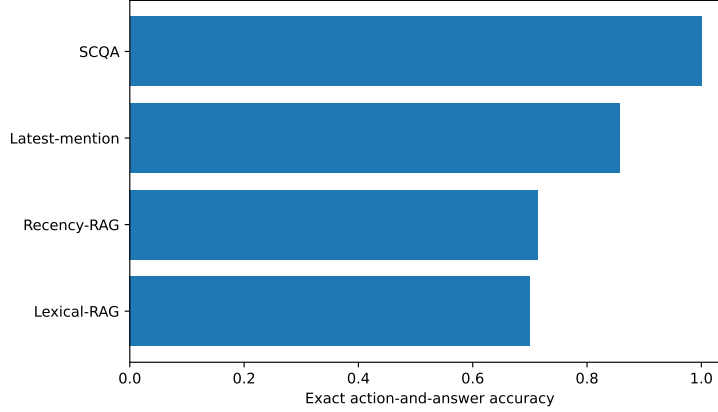


Fig. 3 Oracle-state diagnostic accuracy. The result demonstrates that the specified invariants are executable. It does not estimate end-to-end LLM accuracy because situation variables are supplied rather than inferred.

of the benchmark and implementation and should be interpreted as regression-test coverage, not as estimates of model performance in a population of natural questions.

The category-level pattern provides the useful result. Latest mention fails on hidden-premise items because it answers when clarification is required. Recency ranking fails when the newest global event is not known to the queried observer. Lexical overlap is vulnerable to weak but topically similar conflicting claims. Thus a single aggregate “hallucination” label hides distinct and testable mechanisms (Fig. 4).

Component ablations produce matched failure signatures

Removing temporal validity, source status, modality, scope, observer knowledge, world identity or missing-variable detection selectively damages the matching category (Fig. 5). This causal alignment is the main empirical contribution of the controlled study. It shows that the implementation does not merely benefit from a generic instruction to be cautious; behaviour changes when the specific variable required by an item is removed. Because benchmark and solver share the same schema, the result is best viewed as a mechanistic unit test that future neural systems should pass under both gold and predicted states.

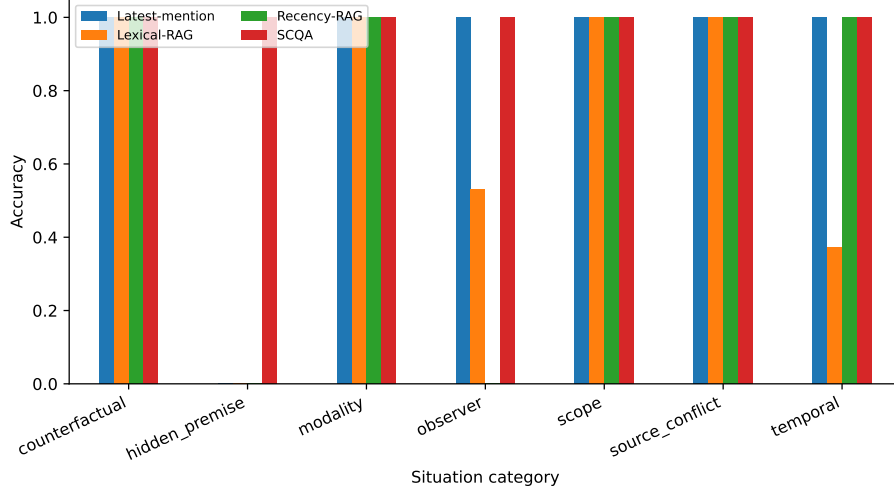


Fig. 4 Accuracy by diagnostic category. Each category isolates one applicability variable, enabling mechanism-level failure localization.

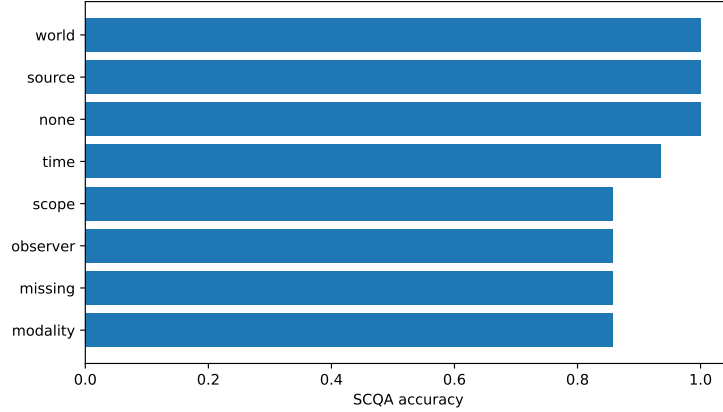


Fig. 5 Oracle-state component ablations. Each removed variable recovers the predicted category-specific failure, providing a causal software diagnostic.

Natural-language benchmarks establish prevalence, not SCQA efficacy

Independent resources show that situation dependence is not confined to the generated benchmark. SituatedQA reported that approximately 16.5% of sampled NQ-Open questions depended on time or place and that updated evidence did not remove a substantial present-time performance gap [6]. FreshQA, PAT-Questions, TimeQA, ChronoQA and TDBench test related problems in freshness, evolving facts and temporal validity [21–25]. These studies motivate the problem definition, but they are not

evaluations of SCQA and are not counted as evidence that the proposed intervention improves a frontier model.

An evidence ladder separates established findings from open claims

Table 3 makes the claim boundary explicit. The current study establishes that seven applicability invariants can be represented, executed and independently ablated under oracle structured states. It supports, but does not establish, the hypothesis that explicit situation states will reduce analogous errors in LLMs. End-to-end efficacy requires matched multi-model experiments on natural text, predicted-state evaluation, strong prompt/RAG/agent baselines, clustered uncertainty estimates, human annotation and efficiency analysis. We release the adapter, schemas and reporting protocol for these experiments but do not substitute unexecuted predictions for results.

Discussion

The core insight is that relevance is not applicability. Retrieval can return a well-supported statement while a system still applies it to the wrong time, scope, epistemic state or possible world. Situation engineering turns this failure from an informal complaint about “missing context” into a set of state variables, update rules and behavioural consequences that can be tested separately.

A complementary layer, not a replacement vocabulary

Prompt, context, retrieval, memory, tool, harness, evaluation, security and serving engineering retain distinct responsibilities. Prompts specify the task; context and retrieval supply evidence; memory preserves observations; tools provide capabilities; harnesses govern workflow and permissions; evaluations measure behaviour; serving systems manage resource constraints. Situation engineering owns the query-relative semantics of the active state and the propagation of unresolved applicability into policy. A useful contract is

$$(C_t, M_t, O_t, H_t) \rightarrow (S_t, U_t, P_t), \quad (6)$$

where context, memory, tool observations and harness traces produce candidate situations S_t , unresolved variables U_t and the permitted policy P_t .

This contract prevents conceptual inflation. Retrieval quality should not be renamed situation engineering, and a JSON field named “situation” is not evidence of situation awareness. A system must show that state representation causally changes behaviour in predicted cases, that state uncertainty affects answering or action, and that failures can be assigned to sensing, update, validation, policy or generation.

Implications for hallucination and agent evaluation

A generated claim can be textually entailed by a document yet wrong for the query. Evaluation should therefore record both support and applicability. In free-form generation, claims should be linked to the situation state and provenance on which they depend. In agents, tools should expose preconditions, state transitions and failure semantics rather than only free-text outputs; harnesses should block consequential actions while answer-critical state is unresolved. Evaluation engineering becomes the sensor layer that checks each transition and not merely the final answer.

The response policy is also part of correctness. Clarification and abstention are not failures when the situation is under-specified. Conversely, excessive clarification is costly and can make systems unusable. Future evaluations should therefore report answer accuracy, clarification utility, unnecessary-question rate, abstention precision, selective risk, coverage, latency, token use and monetary cost. These measures make explicit the trade-off between semantic verification and serving efficiency.

What is established, supported and proposed

This study establishes a formal error class, an interoperable state schema, a reference implementation and matched oracle-state ablations. Independent literature supports the premise that time- and place-dependent questions are common enough to matter. The study does not establish that current frontier models exhibit every proposed category at a particular frequency, nor that SCQA improves them under natural language. Those are hypotheses enabled by the released protocol, not completed results.

This distinction changes the scientific role of the paper. The contribution is not a claim of a production-ready solution. It is a research scaffold that allows subsequent work to ask more precise questions: Is the dominant error in state sensing or state update? Does explicit state outperform a matched structured prompt? Does a temporal graph add value after controlling for token budget? When should a system ask rather than retrieve? How robust is a state to adversarially stale evidence? Which costs arise from maintaining provenance and multiple candidate worlds?

A staged research programme

We propose six stages for cumulative evidence. Stage 1 uses formal and synthetic unit tests to verify semantics. Stage 2 evaluates natural-language situation sensing with slot, event, temporal, scope, modality, observer and world-state metrics. Stage 3 performs matched interventions on several model families while holding evidence and computation budgets constant. Stage 4 tests compositional and distributional generalization: unseen templates, longer event chains, nested scope, conflicting authoritative sources, multiple observers, multilingual data and multimodal observations. Stage 5 adds human annotation and interaction to measure clarification and abstention. Stage 6 evaluates long-running agents, state drift, security attacks, latency, cost and independent auditing.

The corresponding maturity levels are SE0, implicit model-only reasoning; SE1, isolated state tags; SE2, provenance-linked state and update rules; SE3, competing

hypotheses with retrieve/clarify/abstain policies; and SE4, learned multimodal sensing, multi-agent belief tracking, shift monitoring and independent audit. The current reference implementation is SE3 only for its seven supplied oracle variables. It is not SE4 and should not be presented as such.

Limitations and risks

The diagnostic data are generated from a schema shared with the solver, so benchmark-solver circularity is intentional and limits external validity. The item count overstates statistical independence because examples share template families. The baselines are software controls, not state-of-the-art LLM systems. Confidence values are rule-derived and do not constitute neural calibration. No human participants, multilingual evaluation, multimodal sensing, frontier-model calls or deployed-agent study were completed. These limitations are not resolved by additional prose and require new experiments.

Explicit state can also create new attack surfaces. An adversary may manipulate timestamps, source authority, identity, scope or world labels. A centralized state representation may expose sensitive user information or create false confidence in a flawed schema. Situation-engineered systems therefore require provenance, access control, privacy-preserving memory, conflict visibility and independent evaluation. “Explicit” must not be confused with “correct”.

Conclusion

Situation engineering provides a bounded and testable way to connect several communities working on reliable AI. Its durable contribution will depend not on the adoption of a new label, but on whether researchers use the interfaces to produce stronger measurements, interoperable state representations and systems that know when available evidence does and does not apply. The present artefact is intended as the first rung of that evidence ladder: a transparent formal and software scaffold on which natural-language, multi-model and deployment studies can be built.

Methods

Problem formulation

An item consists of a query q , evidence claims E , a query time and, where relevant, an observer or possible-world identifier. A claim may be supported in isolation but inapplicable to the query. We define

$$\text{Apply}(e, q, S) \in \{\text{valid}, \text{invalid}, \text{unresolved}\}, \quad (7)$$

where S is the active situation state. The system first constructs or receives S , then updates competing claims and selects an action from answering, retrieval, clarification, abstention or task-specific action.

The end-to-end system is decomposed as

$$(q, E) \xrightarrow{f_{sense}} \hat{S} \xrightarrow{f_{update}} \hat{S}' \xrightarrow{\pi} a \xrightarrow{g} y. \quad (8)$$

This decomposition supports stage-specific evaluation. The current controlled experiment supplies S directly and evaluates f_{update} , π and the deterministic answer renderer. It does not evaluate f_{sense} on unrestricted language.

Situation schema

The schema contains actors and entities; events and state transitions; valid and observation time; scope; modality and source status; observer-specific knowledge; possible-world identity; provenance; and unresolved answer-critical variables. Update rules include temporal supersession, cancellation, source-authority preference where specified, scope containment, observer visibility and world separation. Unresolved variables are preserved rather than converted into a guessed binary state.

SituationCatch-Bench generation

The deterministic generator creates 600 instances for each of seven categories using seeded lexical variation and shuffled claim order. Each item includes the question, claims, structured situation fields, gold action, gold answer aliases and category. The categories and gold semantics are documented in the bundled datasheet. Generation and evaluation are intentionally co-designed: the benchmark is a conformance suite for explicit semantics, analogous to unit tests for a software protocol, rather than an independent estimate of natural-language performance.

Future benchmark versions should use separate authors for data construction and system development, clustered scenario families, hidden generators, natural and semi-synthetic sources, adjudicated annotations, compositional splits and adversarial state manipulations. Recommended natural-data fields and annotation instructions are supplied in the Supplementary Information.

Reference implementation

SCQA is a deterministic Python reference implementation with no trained parameters. It parses the supplied structured fields, performs category-specific state updates and selects a permitted response. The three software-control baselines select claims using lexical overlap, recency or latest mention and do not maintain the complete situation state. These controls test whether the specified state variables are necessary for the generated invariants; they are not intended to represent competitive LLM, RAG or agent baselines.

Metrics and analysis

The primary diagnostic measure is exact action-and-answer accuracy. For items requiring clarification, correctness requires selecting clarification rather than providing the latent answer. Category-level accuracy and matched component ablations localize failure mechanisms. Because instances share template families, item-level confidence intervals and P values are not used as primary scientific evidence in the revised manuscript. The released scripts retain item-level summaries for regression testing,

but future natural-language studies should use scenario- or template-clustered bootstrap intervals, paired tests, multiple-comparison control and pre-specified primary outcomes.

Rule-derived confidence values from the original implementation are not treated as neural calibration. End-to-end work should report Brier score, expected calibration error, selective risk, risk-coverage area, clarification utility and abstention precision/recall using model- or state-derived probabilities.

Ablation design

Seven ablations independently remove temporal validity, source status, modality, scope, observer knowledge, world identity or missing-variable detection. Each ablation is applied to all items, and its expected effect is specified before execution. The purpose is to test causal alignment between representation and failure category, not to estimate an average treatment effect in natural text.

Protocol for end-to-end multi-model evaluation

A confirmatory study should compare at least the following matched conditions: direct prompting; chain-of-thought or structured prompting; date-aware context; standard RAG; temporal or event-aware RAG; reflection or verification agent; and Situation Engineering. The generation model, evidence set, retrieval corpus, maximum context, output budget and sampling parameters should be held constant where the design permits. At least three proprietary and three open-weight model families should be evaluated using immutable model identifiers.

For each call, researchers should archive the prompt, evidence, raw response, normalized answer, action decision, model identifier, provider, execution date, latency, token use, monetary cost, errors and retries. Evaluation must report gold-state, predicted-state and deliberately corrupted-state conditions. The predicted-state condition should score entity/event extraction, temporal relations, validity intervals, scope, modality, source status, observer knowledge, possible-world identity and final answer. This enables error attribution to sensing, updating, policy or generation.

The bundled `code/run_multimodel_eval.py` provides provider adapters and refuses to create placeholder outputs when credentials are absent. No frontier-model performance values are reported because such calls were not executed in the study environment. The script is an executable protocol, not a completed experiment.

Human and deployment evaluation

Human evaluation should use at least three independent annotators per natural item, blinded to method, with a documented adjudication procedure. Recommended outcomes include state-slot agreement, answer correctness, appropriateness of clarification or abstention, unnecessary clarification and perceived justification quality. Studies involving participants should report ethics approval or exemption, consent, compensation and privacy handling.

Deployment studies should additionally measure state drift, invalidation latency, recovery after contradictory observations, action-precondition violations, security

attacks on state metadata, throughput, tail latency, energy, token use and cost. The state and provenance required to audit a consequential decision should be retained subject to privacy and access-control requirements.

Reproducibility and reporting protocol

The package contains the generator, generated benchmark, reference implementation, predictions, figures, datasheet and exact commands. To make future systems comparable, authors should report: (1) the situation schema; (2) which variables are supplied, retrieved or inferred; (3) update and conflict semantics; (4) response and action policies; (5) provenance and observability; (6) stage-specific errors; (7) prompt, context, retrieval, memory, tool and harness configurations; and (8) resource costs. A machine-readable experiment manifest is recommended.

Ethics and use of generative AI

The controlled benchmark contains no personal records or human participants. Generative AI assisted drafting and software development. The author inspected the code, generated outputs, calculations and references and remains responsible for the manuscript. No empirical values were fabricated to represent unexecuted model calls or human studies.

Data availability

SituationCatch-Bench v0.1 is included in the submission package as `data/situationcatch.bench.jsonl`, together with its deterministic generator, datasheet and item-level outputs. This package is sufficient to reproduce the controlled results. A public archival repository, immutable release, licence and DOI are required before acceptance; no placeholder DOI is presented as an existing deposit.

Code availability

The complete executable implementation is included under `code/`. Running `python code/run_experiments.py --n-per-category 600` regenerates the benchmark, predictions, summaries and ablations without network access or proprietary dependencies. The multi-model adapter deliberately exits when provider credentials are absent and never creates synthetic API results. A public immutable release and archival DOI remain pre-acceptance requirements.

Author Contributions

G.L.: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Competing Interests

The author declares no competing interests.

Acknowledgements

The author thanks colleagues who provided early discussions on reliable language-model reasoning. No external funding is declared in this draft.

References

- [1] Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv. Neural Inf. Process. Syst.* **33**, 9459–9474 (2020).
- [2] Asai, A., Wu, Z., Wang, Y., Sil, A. & Hajishirzi, H. Self-RAG: learning to retrieve, generate, and critique through self-reflection. In *Proc. International Conference on Learning Representations* (2024).
- [3] Yao, S. et al. ReAct: synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations* (2023).
- [4] Schick, T. et al. Toolformer: language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **36** (2023).
- [5] Min, S., Michael, J., Hajishirzi, H. & Zettlemoyer, L. AmbigQA: answering ambiguous open-domain questions. In *Proc. EMNLP*, 5783–5797 (2020).
- [6] Zhang, M. J. Q. & Choi, E. SituatedQA: incorporating extra-linguistic contexts into QA. In *Proc. EMNLP*, 7371–7387 (2021).
- [7] Kamath, A., Jia, R. & Liang, P. Selective question answering under domain shift. In *Proc. ACL*, 5684–5696 (2020).
- [8] Lin, S., Hilton, J. & Evans, O. TruthfulQA: measuring how models mimic human falsehoods. In *Proc. ACL*, 3214–3252 (2022).
- [9] Ji, Z. et al. Survey of hallucination in natural language generation. *ACM Comput. Surv.* **55**, 1–38 (2023).
- [10] Guo, C., Pleiss, G., Sun, Y. & Weinberger, K. Q. On calibration of modern neural networks. In *Proc. ICML*, 1321–1330 (2017).
- [11] Manakul, P., Liusie, A. & Gales, M. J. F. SelfCheckGPT: zero-resource black-box hallucination detection for generative large language models. In *Proc. EMNLP*, 9004–9017 (2023).
- [12] Thorne, J., Vlachos, A., Christodoulopoulos, C. & Mittal, A. FEVER: a large-scale dataset for fact extraction and verification. In *Proc. NAACL-HLT*, 809–819 (2018).
- [13] Allen, J. F. Maintaining knowledge about temporal intervals. *Commun. ACM* **26**, 832–843 (1983).
- [14] Pearl, J. *Probabilistic Reasoning in Intelligent Systems* (Morgan Kaufmann, 1988).
- [15] Wei, J. et al. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **35**, 24824–24837 (2022).
- [16] Mei, L. et al. A survey of context engineering for large language models. *Preprint at* <https://arxiv.org/abs/2507.13334> (2025).
- [17] Zhong, H. & Zhu, S. AI harness engineering: a runtime substrate for foundation-model software agents. *Preprint at* <https://arxiv.org/abs/2605.13357> (2026).
- [18] Lin, J. et al. Agentic harness engineering: observability-driven automatic evolution of coding-agent harnesses. *Preprint at* <https://arxiv.org/abs/2604.25850>

- (2026).
- [19] Barwise, J. & Perry, J. *Situations and Attitudes* (MIT Press, 1983).
 - [20] Madaan, A. et al. Self-refine: iterative refinement with self-feedback. *Adv. Neural Inf. Process. Syst.* **36** (2023).
 - [21] Vu, T. et al. FreshLLMs: refreshing large language models with search engine augmentation. *Preprint at* <https://arxiv.org/abs/2310.03214> (2023).
 - [22] Meem, J. et al. PAT-Questions: a self-updating benchmark for present-anchored temporal question answering. In *Findings of the Association for Computational Linguistics: ACL* (2024).
 - [23] Chen, W., Wang, X., Wang, W. Y. & Chen, W. A dataset for answering time-sensitive questions. In *Proc. NeurIPS Datasets and Benchmarks* (2021).
 - [24] Chen, Z. et al. A question answering dataset for temporal-sensitive retrieval-augmented generation. *Sci. Data* (2025).
 - [25] Kim, S., Wang, J., Xie, X. & Whang, S. E. Harnessing temporal databases for systematic evaluation of factual time-sensitive question answering in large language models. *Preprint at* <https://arxiv.org/abs/2508.02045> (2025).
 - [26] Model Context Protocol contributors. Model Context Protocol specification, revision 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25> (2025).
 - [27] Anthropic. Demystifying evals for AI agents. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents> (2026).
 - [28] OpenAI. A shared playbook for trustworthy third-party evaluations. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/> (2026).
 - [29] Li, B., Jiang, Y., Gadepally, V. & Tiwari, D. LLM inference serving: survey of recent advances and opportunities. *Preprint at* <https://arxiv.org/abs/2407.12391> (2024).
 - [30] Du, P. Memory for autonomous LLM agents: mechanisms, evaluation, and emerging frontiers. *Preprint at* <https://arxiv.org/abs/2603.07670> (2026).
 - [31] McCarthy, J. & Hayes, P. J. Some philosophical problems from the standpoint of artificial intelligence. In *Machine Intelligence 4*, 463–502 (Edinburgh Univ. Press, 1969).
 - [32] Kowalski, R. & Sergot, M. A logic-based calculus of events. *New Gener. Comput.* **4**, 67–95 (1986).
 - [33] Reiter, R. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems* (MIT Press, 2001).
 - [34] Alchourrón, C. E., Gärdenfors, P. & Makinson, D. On the logic of theory change: partial meet contraction and revision functions. *J. Symb. Logic* **50**, 510–530 (1985).
 - [35] Doyle, J. A truth maintenance system. *Artif. Intell.* **12**, 231–272 (1979).
 - [36] Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. Planning and acting in partially observable stochastic domains. *Artif. Intell.* **101**, 99–134 (1998).
 - [37] Henderson, M., Thomson, B. & Williams, J. D. The second dialog state tracking challenge. In *Proc. SIGDIAL*, 263–272 (2014).
 - [38] Snodgrass, R. T. Temporal databases. In *Theories and Methods of Spatio-Temporal Reasoning in Geographic Space*, 22–64 (Springer, 1992).

AI-use disclosure

Generative AI tools assisted language drafting, code scaffolding and editorial revision. The author reviewed all generated text and code, independently checked the reported controlled outputs and references, and takes responsibility for the manuscript. Generative AI was not treated as an author, annotator or source of unexecuted empirical measurements.

Table 2 Complementary engineering disciplines for reliable AI. Boundaries are analytical, and substantial implementation overlap is expected.

Discipline	Primary design object	Reliability question	Characteristic failure if absent
Prompt engineering	Instructions, roles and output constraints	Did the model understand what it was asked to do?	Wrong task, format or reasoning procedure
Context engineering	Evidence payload and context-window allocation	Did the model receive the information needed to do it?	Missing, irrelevant or poorly organized evidence
Retrieval engineering	Query formulation, indexes, ranking and freshness	Was the best available evidence found efficiently?	Relevant evidence omitted or stale evidence over-ranked
Memory engineering	Write, consolidation, forgetting and recall policies	What should persist across turns and sessions?	Lost commitments, repeated mistakes or stale memories
Situation engineering	Time-indexed, scoped, observer-relative world state	Which available facts apply here and now, and what remains unresolved?	Confident use of expired, out-of-scope, hypothetical or unknown facts
Tool / function-calling engineering	Capability schemas, descriptions, pre-conditions and error semantics	Can the model select and invoke the right capability correctly?	Wrong tool, malformed arguments or silent tool failure
Protocol / interface engineering	Interoperability, lifecycle, authorization and capability negotiation	Can components exchange context and actions safely and consistently?	Integration mismatch, privilege confusion or ambiguous contracts
Harness engineering	Orchestration, permissions, retries, checkpoints and recovery	Can the system execute a multi-step workflow safely?	Unchecked actions, brittle loops or unrecoverable execution
Evaluation engineering	Tasks, rubrics, judges, counterfactuals and regression suites	Can reliability claims be measured and falsified?	Hidden regressions, metric gaming or unsupported claims
Observability / provenance engineering	Traces, state snapshots, lineage and audit records	Can an operator reconstruct why the output occurred?	Un-debuggable failures and unverifiable decisions
Inference / serving engineering	Scheduling, batching, caching, placement and numerical execution	Can the system meet latency, throughput, cost and energy constraints?	Timeouts, overload, excessive cost or quality loss under load
Guardrail / security engineering	Policy enforcement, isolation, authorization and abuse resistance	Are actions and data flows constrained to acceptable risk?	Unsafe action, leakage, injection or privilege escalation

Table 3 Evidence ladder and permitted interpretation.

Evidence level	What is available	Permitted claim
E1: formal definition	Situation schema and applicability relation	The failure class is operationally testable
E2: oracle unit tests	4,200 generated instances and matched ablations	Specified invariants are executable and causally separable
E3: natural prevalence	Independent temporal/situated QA literature	Situation dependence occurs in natural questions
E4: predicted-state models	Not completed in this study	Needed to measure sensing and update error
E5: multi-model intervention	Execution harness only; no credentials in the study environment	Needed to claim LLM improvement and generality
E6: deployment / human study	Not completed	Needed for operational safety and human-level comparisons